# Biometria - Projekt 1

Julia Radacka, Paula Wołkowska

1 kwietnia 2026

## 1 Wstęp

Celem projektu było stworzenie aplikacji okienkowej do edycji i analizy obrazów rastrowych. Głównym założeniem było połączenie klasycznych metod przetwarzania obrazu z narzędziami pozwalającymi na ich diagnostykę. Zaimplementowane funkcjonalności obejmują operacje punktowe, filtry splotowe oraz detekcję krawędzi. Program pozwala również na generowanie histogramu oraz wyznaczanie projekcji poziomej i pionowej. W dalszej części raportu opisano budowę aplikacji, zastosowane algorytmy oraz przedstawiono wyniki testów.

## 2 Opis aplikacji

Program został zaimplementowany w języku Java jako aplikacja desktopowa z interfejsem graficznym opartym na bibliotece Swing. Do zarządzania projektem i jego budowy wykorzystano narzędzie Maven. Główny moduł aplikacji stanowi edytor umożliwiający wczytanie obrazów, ich przetwarzanie w czasie rzeczywistym oraz zapis wyników do pliku. W menu „Operacje” zgromadzone zostały wszystkie dostępne przekształcenia. Interfejs został zaprojektowany w sposób dynamiczny. Operacje takie jak negatyw czy gotowe filtry splotowe wykonywane są natychmiastowo, natomiast funkcje wymagające dodatkowych danych (np. próg binaryzacji czy parametry własnej maski) wywołują dedykowane okna dialogowe do wprowadzania parametrów.
Ponadto aplikacja umożliwia wyświetlenie histogramu, a także wygenerowanie projekcji poziomej i pionowej dla analizowanego obrazu.

## 3 Opis metod i prezentacja wyników działania

W rozdziale przedstawiono zasady działania zaimplementowanych w aplikacji metod oraz przykłady ich praktycznego zastosowania.

### 3.1 Operacje punktowe

Operacje punktowe modyfikują każdy piksel niezależnie od pozostałych, opierając się na funkcji przekształcenia wartości kanałów barwnych lub jasności. W zaimplementowanych algorytmach, o ile nie zaznaczono inaczej, kanał alfa (przezroczystość) pozostaje niezmieniony, a obliczenia dotyczą składowych $R, G, B$. W przypadku operacji wykraczających

poza nominalny zakres $[0, 255]$, stosowana jest funkcja nasycenia:

$$\text{clamp}(x) = \max(0, \min(255, x))$$

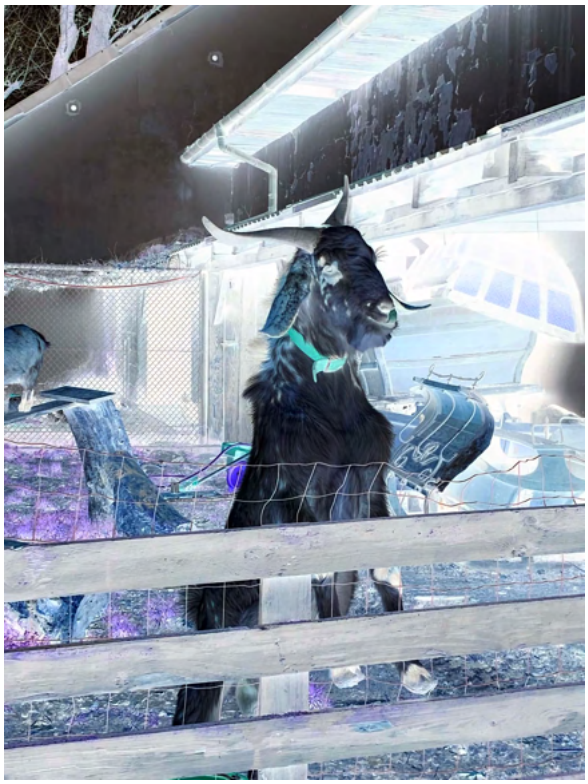
### 3.1.1 Negatyw

Operacja polega na odwróceniu poziomów jasności składowych barwnych. Dla każdego kanału obliczana jest wartość dopełniająca do maksimum:

$$r' = 255 - r \tag{1}$$
$$g' = 255 - g \tag{2}$$
$$b' = 255 - b \tag{3}$$

Efektem jest zamiana obszarów jasnych na ciemne, co ułatwia m.in. wizualizację szczegółów w głębokich cieniach obrazu. Zastosowanie algorytmu w praktyce można zaobserwować na Rysunku 2.

Rysunek 1: Obraz wejściowy



Rysunek 2: Obraz w negatywie

### 3.1.2 Korekta jasności

Polega na dodaniu stałej wartości przesunięcia $\Delta$ (parametru) do składowych RGB. Nowe wartości wyznaczane są według wzorów:

$$r' = clamp(r + \Delta) \tag{4}$$
$$g' = clamp(g + \Delta) \tag{5}$$
$$b' = clamp(b + \Delta) \tag{6}$$

Zbyt wysokie wartości dodatnie $\Delta$ prowadzą do przepalenie jasnych fragmentów obrazu (wartości są obcinane do 255), natomiast zbyt duże ujemne wartości $\Delta$ prowadzą do utraty informacji w cieniach (wartości są obcinane do 0).
W celu zaprezentowania działania algorytmu na wybranym obrazie testowym, parametr przesunięcia $\Delta$ ustawiono na wartość dodatnią równą 94. Taka konfiguracja skutkuje globalnym rozjaśnieniem obrazu, pozwalając na uwydatnienie detali ukrytych w ciemniejszych partiach fotografii.

Rysunek 3: Obraz wejściowy

Rysunek 4: Obraz po przetworzeniu

### 3.1.3 Korekta kontrastu

Operacja ta ma na celu zwiększenie lub zmniejszenie różnic w jasności pikseli. Na podstawie parametru *contrast* wyznaczany jest współczynnik wzmocnienia:

$$factor = \frac{259 \cdot (contrast + 255)}{255 \cdot (259 - contrast)} \tag{7}$$

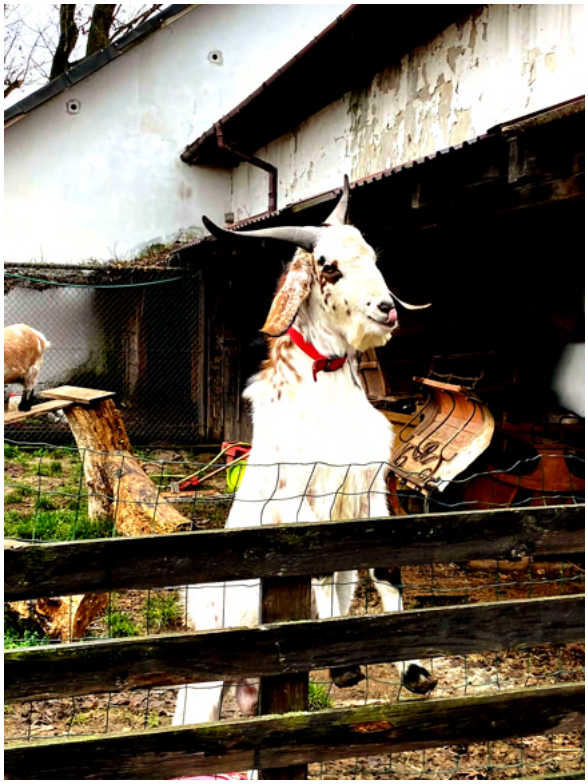
Nowe składowe obliczane są z przesunięciem względem środka zakresu (128):

$$newR = clamp(\lfloor factor \cdot (R - 128) + 128 \rfloor) \tag{8}$$
$$newG = clamp(\lfloor factor \cdot (G - 128) + 128 \rfloor) \tag{9}$$
$$newB = clamp(\lfloor factor \cdot (B - 128) + 128 \rfloor) \tag{10}$$

Dla dodatnich wartości parametru kontrastu różnice pomiędzy jasnymi i ciemnymi obszarami stają się większe, natomiast dla mniejszych wartości efekt jest odwrotny (spłaszczenie kontrastu). Zbyt duże wzmocnienie kontrastu może prowadzić do klipowania wartości do 0 lub 255, a w konsekwencji do utraty szczegółów.

Do celów demonstracyjnych operacji przedstawionych na Rysunku 6 wybrano parametr kontrastu o dokładnej wartości 75. Zastosowanie tak zdefiniowanego, dodatniego współczynnika powoduje zauważalne uwydatnienie różnic między jasnymi i ciemnymi obszarami. Obraz staje się bardziej wyrazisty, a jego dynamika tonalna ulega rozszerzeniu.

Rysunek 5: Obraz wejściowy



Rysunek 6: Obraz po przetworzeniu

### 3.1.4 Binaryzacja (progowanie)

Binaryzacja przekształca obraz do postaci czarno-białej na podstawie progu *threshold*. Najpierw wyznaczana jest luminancja (ważona suma RGB):

$$luminance = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B \tag{11}$$

a następnie przypisywana jest wartość binarna:

$$\text{newVal} = \begin{cases} 255 & \text{jeżeli luminance} \geq \text{threshold} \\ 0 & \text{jeżeli luminance} < \text{threshold} \end{cases} \tag{12}$$

Wartość $newVal$ jest przypisywana do wszystkich trzech kanałów, co daje obraz czarno-biały (0 lub 255). W tej operacji wynikowy piksel staje się w pełni nieprzezroczysty ($\alpha = 255$). Metoda jest wrażliwa na dobór progu oraz na nierównomierne oświetlenie. Dla różnych obrazów lub różnych fragmentów obrazu optymalny próg może być inny.

Rysunek 7: Obraz wejściowy

Rysunek 8: Obraz po przetworzeniu

Na Rysunku 8 widać efekt zastosowania binaryzacji, na którym próg odcięcia (*threshold*) ustalono na poziomie 128.

### 3.1.5 Wyrównanie histogramu

Operacja ta służy do automatycznej poprawy kontrastu poprzez przekształcenie wartości pikseli na podstawie dystrybuanty skumulowanej (CDF). Proces realizowany jest niezależnie dla każdego kanału $(R, G, B)$ w następujących krokach:

1. Zliczenie histogramu wystąpień wartości $[0, 255]$.
2. Obliczenie dystrybuanty skumulowanej $cdf(v)$, będącej sumą narastającą histogramu.
3. Wyznaczenie pierwszej niezerowej wartości $cdfMin$, co pozwala na prawidłowe skalowanie w przypadku obrazów o ograniczonej rozpiętości tonalnej.

Nowe wartości kanałów obliczane są według zależności:

$$newR = \text{round}\left(\frac{cdf[R] - cdfMin_R}{N - cdfMin_R} \cdot 255\right) \tag{13}$$

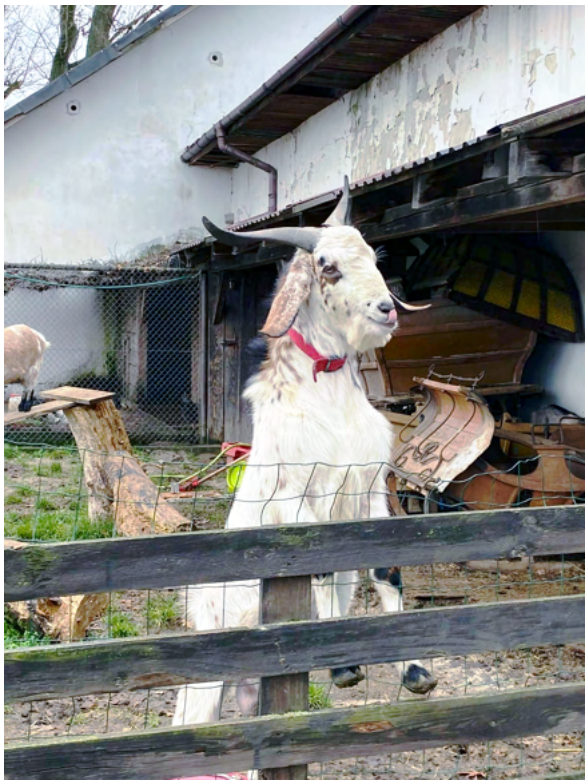
$$newG = \text{round}\left(\frac{cdf[G] - cdfMin_G}{N - cdfMin_G} \cdot 255\right) \tag{14}$$

$$newB = \text{round}\left(\frac{cdf[B] - cdfMin_B}{N - cdfMin_B} \cdot 255\right) \tag{15}$$

gdzie $N$ oznacza całkowitą liczbę pikseli obrazu. Metoda ta skutecznie poprawia widoczność szczegółów w obrazach o niskim kontraście, choć niezależne przetwarzanie kanałów

może prowadzić do zmiany pierwotnej kolorystyki. W tej operacji kanał alfa jest ustawiany na wartość 255.
Efekty zastosowania metody widać na Rysunku 10.

Rysunek 9: Obraz wejściowy



Rysunek 10: Obraz po przetworzeniu

### 3.1.6 Odcienie szarości - Luminancja

Metoda ta przekształca obraz kolorowy do skali szarości, uwzględniając fizjologię ludzkiego oka i jego nieliniową wrażliwość na różne barwy (największą na kolor zielony). Wartość jasności wyznaczana jest jako ważona suma składowych:

$$gray = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B \tag{16}$$

Wyznaczona wartość jest przypisywana do wszystkich trzech kanałów barwnych, co pozwala na zachowanie oryginalnego kanału alfa i uzyskanie naturalnie wyglądającej skali szarości.
Na Rysunku 12 można zobaczyć efekt zastosowania metody.

Rysunek 11: Obraz wejściowy

Rysunek 12: Obraz po przetworzeniu

### 3.1.7 Odcienie szarości - Średnia RGB

Jest to uproszczona metoda konwersji, w której poziom szarości stanowi średnia arytmetyczna składowych barwnych:

$$avg = \frac{R + G + B}{3} \tag{17}$$

Metoda ta nie uwzględnia różnic w percepcji barw przez człowieka. Wynikowy piksel otrzymuje nową wartość $avg$ dla kanałów RGB oraz pełną nieprzezroczystość ($\alpha = 255$). Przykład zastosowania metody widoczny jest na Rysunku 14.

Rysunek 13: Obraz wejściowy

Rysunek 14: Obraz po przetworzeniu

### 3.1.8 Odcienie szarości - Jasność (Rozstęp)

Metoda ta wyznacza poziom szarości na podstawie uśrednionego rozstępu skrajnych wartości składowych RGB. Dla każdego piksela wyznaczane są wartości ekstremalne:

$$max = \max(R, G, B) \tag{18}$$

$$min = \min(R, G, B) \tag{19}$$

a następnie obliczana jest ich średnia:

$$lightness = \frac{max + min}{2} \tag{20}$$

Wartość *lightness* jest wpisywana do kanałów $R, G$ i $B$, tworząc piksel w skali szarości. Podobnie jak w przypadku średniej RGB, wynikowy obraz jest w pełni nieprzezroczysty, a metoda służy do szybkiego przybliżenia jasności na podstawie dynamiki kanałów wejściowych.
Przykład zastosowania odcieni szarości (jasności) widoczny jest na Rysunku 16.

Rysunek 15: Obraz wejściowy

Rysunek 16: Obraz po przetworzeniu

## 3.2 Operacje oparte o sąsiedztwo (filtry)

Operacje te, w przeciwieństwie do punktowych, wyznaczają nową wartość piksela na podstawie analizy jego otoczenia. Pozwala to na realizację zadań takich jak wygładzanie, wyostrzanie czy ekstrakcja krawędzi.

### 3.2.1 Konwolucja

Mechanizm konwolucji polega na obliczeniu sumy ważonej wartości pikseli z otoczenia punktu $(x, y)$ zgodnie z zadaną maską (jądrem) filtru. W aplikacji zaimplementowano uniwersalny silnik splotowy, który dla każdego kanału $(R, G, B)$ wykonuje operację:

$$V_{out} = \text{clamp}\left(\frac{\sum_{i,j} \text{maska}(i,j) \cdot V_{in}(i,j)}{weight}\right), \quad V \in \{R, G, B\} \tag{21}$$

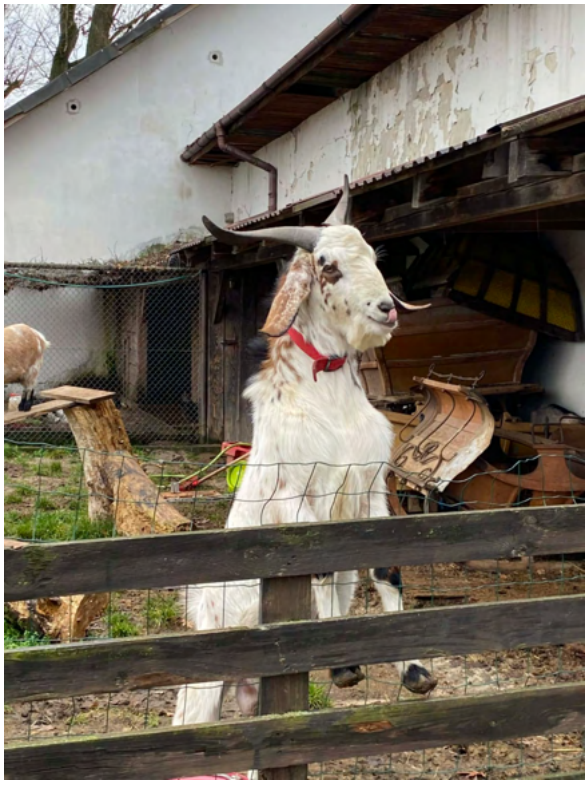
Gdzie $V_{in}(i, j)$ oznacza wartość odpowiedniego kanału piksela z sąsiedztwa objętego maską, a $weight$ pełni rolę współczynnika normalizacyjnego. W procesie tym kanał alfa $(\alpha)$ pozostaje niezmieniony i jest kopiowany bezpośrednio z piksela wejściowego.

W implementacji zastosowano także prostą obsługę brzegów obrazu: dla pikseli znajdujących się zbyt blisko krawędzi (tam, gdzie maska wychodziłaby poza obraz) aplikacja nie liczy konwolucji, tylko przepisuje piksel wejściowy do obrazu wynikowego bez zmian. Oznacza to, że filtr działa w pełni na obszarze wewnętrznym obrazu, natomiast brzegi pozostają takie jak w oryginale.

### 3.2.2 Filtr uśredniający

Jest to najprostszy filtr wygładzający (dolnoprzepustowy). Wykorzystuje maskę $3 \times 3$ wypełnioną jedynkami oraz wagę $weight = 9$. Operacja redukuje szum wysokoczęstotliwościowy, jednak skutkiem ubocznym jest widoczne rozmycie krawędzi obiektów.
Przykład zastosowania filtru widoczny jest na Rysunku 18.

Rysunek 17: Obraz wejściowy



Rysunek 18: Obraz po przetworzeniu

### 3.2.3 Filtr Gaussa

Stanowi zaawansowaną alternatywę dla filtru uśredniającego. Dzięki zastosowaniu maski opartej na dyskretnym rozkładzie Gaussa, piksele znajdujące się bliżej centrum mają większy wpływ na wynik końcowy:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, weight = 16 \tag{22}$$

Pozwala to na uzyskanie bardziej naturalnego efektu wygładzenia przy mniejszej degradacji ostrości w porównaniu do zwykłego uśredniania. Efekt filtru widoczny jest na Rysunku 20.

Rysunek 19: Obraz wejściowy

Rysunek 20: Obraz po przetworzeniu

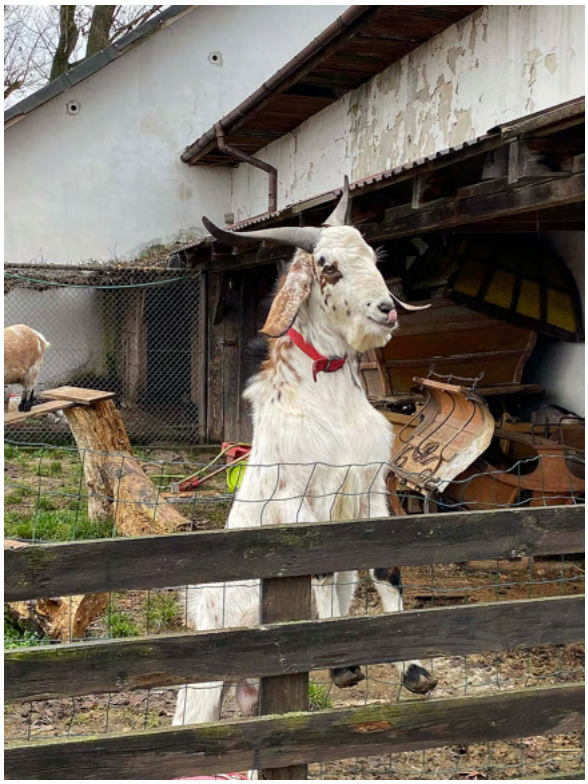
### 3.2.4 Wyostrzanie

Filtr ten (górnoprzepustowy) wzmacnia różnice lokalne między sąsiednimi pikselami. Wykorzystuje maskę:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, weight = 1 \tag{23}$$

Zwiększenie wartości centralnej przy jednoczesnym odjęciu wartości sąsiadów uwypukla krawędzie i drobne detale, choć może również uwidocznić istniejący w obrazie szum. Przykład zastosowania tej metody można zaobserwować na Rysunku 22.

Rysunek 21: Obraz wejściowy



Rysunek 22: Obraz po przetworzeniu

### 3.2.5 Filtr własny

Aplikacja umożliwia użytkownikowi zdefiniowanie własnej maski konwolucji. Użytkownik podaje wartości maski oraz parametr *weight*, który służy do normalizacji wyniku. Funkcjonalność ta pozwala na tworzenie niestandardowych efektów splotowych.
W ramach testowania tej funkcjonalności zdefiniowano następującą maskę:

$$M = \begin{bmatrix} -3 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{bmatrix} \tag{24}$$

Efekty jej zastosowania widać na Rysunku 24.

Rysunek 23: Obraz wejściowy

Rysunek 24: Obraz po przetworzeniu

### 3.2.6 Detekcja krawędzi – operator Sobela

Operator Sobela to jeden z powszechnie stosowanych algorytmów detekcji krawędzi, bazujący na dyskretnej aproksymacji gradientu dwuwymiarowej funkcji jasności obrazu. Istota metody sprowadza się do zastosowania dwóch masek splotowych o wymiarach $3 \times 3$, służących do estymacji składowych wektora gradientu w kierunku poziomym ($G_x$) oraz pionowym ($G_y$):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{25}$$

Ostateczna amplituda gradientu (moduł wektora), określająca reprezentację siły krawędzi w danym pikselu, wyznaczana jest na podstawie euklidesowej miary odległości:

$$M = \sqrt{G_x^2 + G_y^2} \tag{26}$$

**Zastosowanie i ograniczenia:**
Warunkiem wstępnym dla prawidłowego funkcjonowania omawianego filtru jest konwersja obrazu wejściowego do skali szarości. Ponadto, wyznaczone wartości gradientu mogą przekraczać standardowy, 8-bitowy zakres jasności. Z tego względu niezbędne jest zastosowanie funkcji nasycenia (ang. *clamp*), która ogranicza wyniki do maksymalnej dopuszczalnej wartości.
Kluczowym atutem operatora Sobela jest inherentna filtracja dolnoprzepustowa (uśrednianie), wynikająca z obecności wag 2 i -2 w macierzach splotowych. Cecha ta zapewnia algorytmowi stosunkowo wysoką odporność na szum. W rezultacie wyekstrahowane krawędzie są szersze, bardziej ciągłe i wyraźnie zarysowane. Należy również zaznaczyć, że z

uwagi na wymiary maski, algorytm w swojej podstawowej formie pomija analizę pikseli znajdujących się na skrajnych krawędziach obrazu.

Rysunek 25: Obraz wejściowy

Rysunek 26: Obraz po przetworzeniu

Działanie algorytmu w praktyce zilustrowano na Rysunku 26. Zauważalne są wyraźne, pogrubione kontury przy jednoczesnym tłumieniu drobnych zakłóceń tła.

### 3.2.7 Detekcja krawędzi – krzyż Robertsa (operator Robertsa)

Operator krzyżowy Robertsa stanowi jedną z najwcześniejszych i obliczeniowo najprostszych metod detekcji krawędzi. Algorytm ten wykorzystuje dwie niewielkie maski splotowe o wymiarach $2 \times 2$, które aproksymują pierwszą pochodną w kierunkach diagonalnych (pod kątem 45 oraz 135 stopni):

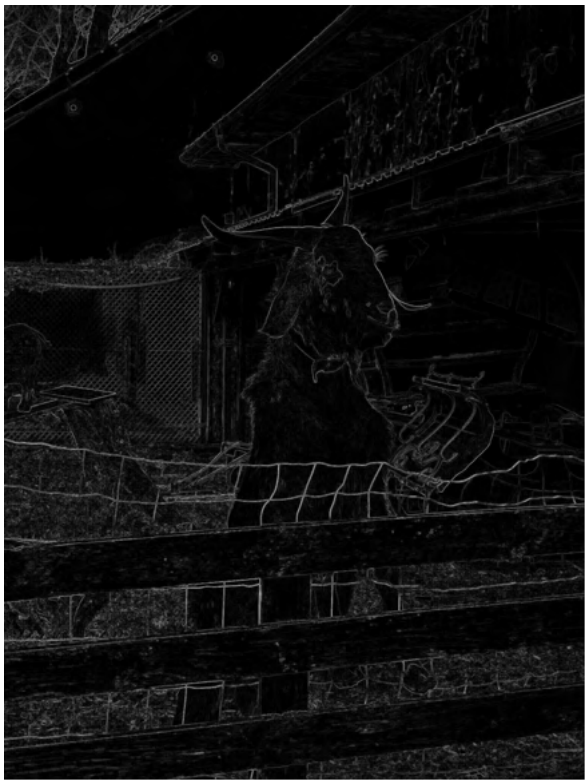
$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \tag{27}$$

Przez analogię do metody Sobela, wypadkowa wartość jasności krawędzi definiowana jest jako pierwiastek z sumy kwadratów składowych $G_x$ i $G_y$.

**Zastosowanie i ograniczenia:**

Główną zaletą metody jest wysoka wydajność obliczeniowa oraz zdolność do ekstrakcji niezwykle cienkich, w tym jednopikselowych, krawędzi. Niemniej jednak, brak jakiegokolwiek mechanizmu uśredniającego w maskach splotowych skutkuje wysoką podatnością algorytmu na zakłócenia, w szczególności na szum impulsowy (tzw. szum pieprz i sól). W przypadku przetwarzania obrazów o złożonej teksturze bądź niskiej jakości, operator Robertsa może generować wyniki obarczone dużą liczbą fałszywych detekcji, błędnie interpretując zaszumienie jako rzeczywiste krawędzie.

Rysunek 27: Obraz wejściowy



Rysunek 28: Obraz po przetworzeniu

Na Rysunku 28 zobrazowano efekt zastosowania tej metody. Wyekstrahowane krawędzie są znacznie cieńsze, jednak obraz wynikowy charakteryzuje się podwyższonym poziomem zaszumienia.

## 3.3 Operacje morfologiczne

Operacje morfologiczne stanowią fundamentalną gałąź przetwarzania obrazów binarnych, mającą zastosowanie w analizie kształtów, usuwaniu szumu oraz wzmacnianiu struktur o określonej morfologii.

W niniejszym projekcie zaimplementowano cztery podstawowe operacje morfologiczne: erozję, dylatację, otwarcie oraz zamknięcie. Wszystkie operacje pracują na obrazach binarnych, dlatego przed zastosowaniem którejkolwiek z nich obraz wejściowy jest automatycznie konwertowany do postaci binarnej z progiem $threshold = 128$. Zgodnie z konwencją przyjętą na wykładzie, obiekty na obrazie (obszary zainteresowania) są reprezentowane przez czarne piksele, a tło przez białe. Dla operacji erozji i dylatacji istnieje możliwość stosowania tych operacji wielokrotnie poprzez parametr liczby iteracji, co pozwala na zwiększenie intensywności efektu.

### 3.3.1 Element strukturalny

Kluczowym parametrem dla każdej operacji morfologicznej jest **element strukturalny (SE)** — zbiór współrzędnych definiujący sąsiedztwo, w obrębie którego będą analizowane sąsiady danego piksela.

Rozmiar SE determinuje zasięg operacji morfologicznej. Większy rozmiar SE prowadzi do bardziej drastycznych zmian, natomiast mniejszy SE pozwala na bardziej precyzyjne operacje. Wybór właściwego SE jest niezwykle istotny dla uzyskania zadowalających rezultatów. Implementacja umożliwia użytkownikowi wybór rozmiaru nieparzystego SE od 3 do 21.

Implementacja umożliwia na generowanie pięć odmiennych kształtów SE, parametryzowanych rozmiarem $n$:

- **Prostokątny (RECT)** – wszystkie piksele w otoczeniu $n \times n$ należą do SE; wariant ten zapewnia największy obszar oddziaływania operacji.
- **Krzyżowy (CROSS)** – SE obejmuje wyłącznie piksele leżące na poziomej i pionowej osi przechodzącej przez środek; wariant optymalny do przetwarzania struktur wydłużonych.
- **Kołowy (ELLIPSE)** – aproksymacja koła wpisanego w kwadrat, zdefiniowana warunkiem $dx^2 + dy^2 \leq r^2$, gdzie $r = \lfloor n/2 \rfloor$; wariant ten jest najbardziej zbliżony do operacji izotropowych.
- **Poziomy (HORIZONTAL)** – SE zawiera jedynie piksele w linii poziomej; przydatny do detekcji struktur horyzontalnych.
- **Pionowy (VERTICAL)** – SE zawiera jedynie piksele w linii pionowej; wykorzystywany do detekcji struktur wertykalnych.

Podczas analizy sąsiedztwa dla każdego piksela obrazu zachodzi konieczność odpowiedniego traktowania pikseli brzegowych. W tym celu zastosowano strategię *edge clamping*. Polega ona na ograniczeniu współrzędnych elementu strukturalnego do fizycznych wymiarów obrazu – jeżeli maska wychodzi poza jego obszar, do obliczeń pobierana jest wartość najbliższego dostępnego piksela leżącego na krawędzi. Dzięki temu żaden piksel nie zostaje pominięty, a algorytm nie odwołuje się do pamięci poza obszarem obrazu.

### 3.3.2 Dylatacja

Dylatacja jest operacją morfologiczną, która rozszerza czarne obiekty na obrazie. Stanowi morfologiczną transformację, która łączy dwa zbiory za pomocą wektorowego dodawania. Jest to operacja rozszerzenia zbioru podstawowego $A$ o element strukturalny $B$. Formalnie definiuje się ją jako:

$$A \oplus B = \{p : p = a + b, a \in A, b \in B\} \tag{28}$$

Równoważnie, dylatację można zdefiniować jako połączenie wszystkich przesuniętych punktów obiektu $A$ stosownie do struktury obiektu $B$:

$$A \oplus B = \bigcup_{b \in B} A_b \tag{29}$$

gdzie $A_b$ oznacza obiekt $A$ przesunięty o wektor $b$. W praktyce algorytmicznej, dla każdego piksela wyjściowego badane jest otoczenie zdefiniowane przez element strukturalny. Jeśli **co najmniej jeden** piksel z tego sąsiedztwa należy do obiektu (jest czarny), to piksel

wynikowy staje się czarny, co matematycznie realizuje się poprzez znalezienie wartości minimalnej (0) w analizowanym oknie.

Rysunek 29: Obraz wejściowy (po binaryzacji)

Rysunek 30: Obraz po dylatacji

Rysunek 30 prezentuje rezultat dwóch iteracji operacji dylatacji z użyciem krzyżowego elementu strukturalnego o rozmiarze 3×3.

### 3.3.3 Erozja

Erozja stanowi operację morfologiczną przeciwstawną do dylatacji, zmniejszającą czarne obiekty na obrazie. Definiuje się ją w następujący sposób:

$$A \ominus B = \{p : B_p \subseteq A\} \tag{30}$$

lub alternatywnie:

$$A \ominus B = \bigcap_{b \in B} A_{-b} \tag{31}$$

gdzie $B_p$ to element strukturalny przesunięty do punktu $p$. W implementacji oznacza to, że dla każdego piksela wyjściowego sprawdzane są wszystkie piksele w obrębie elementu strukturalnego. Jeśli **wszystkie** piksele z sąsiedztwa należą do czarnego obiektu, to piksel wynikowy pozostaje czarny; w przeciwnym razie przyjmuje wartość białego tła.
Głównym ograniczeniem erozji jest jej tendencja do całkowitego wyeliminowania drobnych struktur, które mogą być ważne dla analizy. Wielokrotne zastosowanie erozji (poprzez zwiększenie liczby iteracji) znacznie zmniejsza obiekty i może prowadzić do ich fragmentacji lub całkowitego zanikania.

Rysunek 31: Obraz wejściowy (po binaryzacji)

Rysunek 32: Obraz po erozji

Na Rysunku 32 przedstawiono efekt dwukrotnego zastosowania operacji erozji z użyciem krzyżowego elementu strukturalnego o rozmiarze 3×3.

### 3.3.4 Otwarcie (Opening)

Otwarcie jest złożoną operacją morfologiczną będącą kombinacją erozji następowanej dylatacją, przy użyciu tego samego elementu strukturalnego $B$:

$$A \circ B = (A \ominus B) \oplus B \tag{32}$$

Kombinacja tych dwóch operacji pozwala na usunięcie małych czarnych obiektów oraz drobnych wyrostków, przywracając jednak rozmiar większych struktur blisko do rozmiaru wejściowego. Erozja najpierw eliminuje drobne elementy, a następna dylatacja przywraca powiększone struktury do oryginalnego (lub podobnego) rozmiaru. Dzięki temu otwarcie jest szczególnie efektywne w eliminowaniu szumu przy jednoczesnym zachowaniu kształtów większych obiektów.

Operacja ta nie powinna być stosowana bezwarunkowo — jeśli drobne detale stanowią ważną część informacji zawartej w obrazie, otwarcie może doprowadzić do ich utraty.

Rysunek 33: Obraz wejściowy (po binaryzacji)

Rysunek 34: Obraz po otwarciu

Rysunek 34 ilustruje rezultat operacji otwarcia, przeprowadzonej przy użyciu kwadratowego elementu strukturalnego o rozmiarze 5×5.

### 3.3.5 Zamknięcie (Closing)

Zamknięcie jest operacją morfologiczną, stanowiącą odwrotność otwarcia. Polega na dylatacji następowanej erozją, przy użyciu tego samego elementu strukturalnego $B$:

$$A \bullet B = (A \oplus B) \ominus B \tag{33}$$

Dylatacja najpierw rozszerza czarne obiekty (łącząc odseparowane fragmenty), a następnie erozja przywraca struktury do oryginalnego rozmiaru. W wyniku tej operacji małe białe otwory (szum tła) wewnątrz czarnych obiektów zostają wypełnione. Zamknięcie jest zatem komplementarne wobec otwarcia i wykazuje dokładnie odwrotne własności.

Jak w przypadku otwarcia, zamknięcie również wiąże się z potencjalnym ryzykiem utraty szczegółów - jeśli białe otwory stanowią istotne informacje, operacja ta doprowadzi do ich całkowitego zatarcia.

Rysunek 35: Obraz wejściowy (po binaryzacji)

Rysunek 36: Obraz po zamknięciu

Na Rysunku 36 przedstawiono efekt zastosowania operacji zamknięcia z wykorzystaniem kwadratowego elementu strukturalnego o rozmiarze 5×5.

# 4 Wnioski

Realizacja projektu oraz analiza otrzymanych wyników pozwoliła na sformułowanie następujących wniosków, odnoszących się do natury cyfrowego przetwarzania obrazów:

1. **Napotkane problemy i ograniczenia matematyczne:** Jednym z problemów implementacyjnych okazało się przekraczanie przez wartości pikseli dozwolonego zakresu barwnego $[0, 255]$. W operacjach takich jak korekta jasności, kontrastu czy wyostrzanie splotowe, wyniki równań często przyjmowały wartości ujemne lub przekraczające 255. Wymagało to zastosowania funkcji ucinającej (*clamp*). Ciekawym problemem badawczym okazał się również algorytm wyrównywania histogramu – ze względu na dyskretną naturę obrazów cyfrowych (brak możliwości rozszczepienia pikseli o tej samej jasności początkowej), wynikowy wykres nie jest idealnie płaski, lecz przyjmuje poszarpaną formę (tzw. efekt grzebienia). Nie jest to błąd implementacji, lecz fizyczne ograniczenie świata cyfrowego.

2. **Brak uniwersalności pojedynczych metod:** Żadna z zaimplementowanych metod nie działa idealnie dla każdego przypadku. Przykładowo, prosta binaryzacja ze sztywnym progiem (np. 128) całkowicie zawodzi w przypadku obrazów o nierównomiernym oświetleniu (np. zacienionych z jednej strony). Z kolei wyrównywanie histogramu, choć świetnie radzi sobie ze zdjęciami o niskim kontraście, zaaplikowane na

poprawnie naświetlonym zdjęciu może doprowadzić do nienaturalnego, sztucznego „przeostrzenia” kolorów i utraty zamierzonego efektu fotografii.

3. **Rola i zakresy parametrów wejściowych:** Większość parametrów nie może być dobierana w sposób całkowicie dowolny. W operacjach punktowych próg binaryzacji ma sens matematyczny wyłącznie w domkniętym przedziale $[0, 255]$. Parametry jasności i kontrastu w teorii mogą przyjmować dowolne liczby całkowite, jednak w praktyce wartości poza zakresem $[-255, 255]$ prowadzą do całkowitego zniszczenia informacji (całkowicie czarny lub biały obraz). Z kolei w filtrach splotowych kluczowy jest dobór wagi (dzielnika). Ustawienie go na wartość 0 doprowadziłoby do błędu dzielenia przez zero, a wagi nieproporcjonalne do sumy elementów maski powodują nienaturalne ściemnienie lub rozjaśnienie całego obrazu.

4. **Zasadność stosowania różnych wersji algorytmów:** Projekt ewidentnie udowodnił, że posiadanie różnych wersji tego samego algorytmu jest konieczne, gdyż każda z nich ma inne przeznaczenie.

   - **Skala szarości:** Wersja uśredniająca RGB jest najszybsza, ale algorytm bazujący na luminancji daje znacznie lepsze wizualnie rezultaty, ponieważ uwzględnia nieliniową czułość ludzkiego oka (które jest najbardziej wyczulone na barwę zieloną).
   - **Detekcja krawędzi:** Krzyż Robertsa jest szybki i potrafi wykryć idealnie cienkie linie, ale z powodu braku wygładzania wyłapuje również każdy szum na zdjęciu. Operator Sobela, dzięki większej masce $3 \times 3$ z wbudowanym uśrednianiem (wartości 2 i -2), jest znacznie bardziej odporny na zakłócenia i daje lepsze, ciągłe kontury na rzeczywistych fotografiach.

# 5 Bibliografia

# Literatura


[1] Materiały wykładowe oraz instrukcje laboratoryjne do przedmiotu.

[2] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4. wydanie, Pearson, 2018.

[3] W. Burger, M. J. Burge, *Digital Image Processing: An Algorithmic Introduction Using Java*, Springer, 2016.

[4] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2022.

[5] Dokumentacja techniczna Oracle, *Java Platform, Standard Edition – Java 2D API & Swing*, Dostępne online: `https://docs.oracle.com/en/java/javase/`.